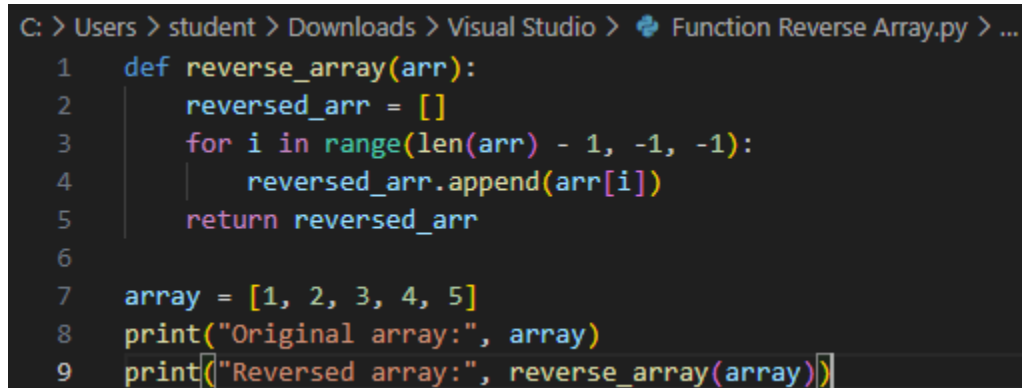
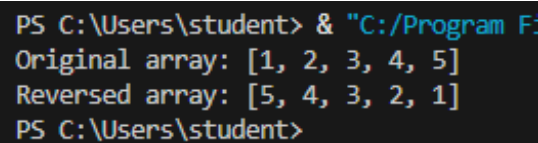


DSA Midterm**1. Write a function that reverses an array without using built-in reverse functions****Code:**

```
def reverse_array(arr):  
    reversed_arr = []  
    for i in range(len(arr) - 1, -1, -1):  
        reversed_arr.append(arr[i])  
    return reversed_arr  
  
array = [1, 2, 3, 4, 5]  
print("Original array:", array)  
print("Reversed array:", reverse_array(array))
```



```
C: > Users > student > Downloads > Visual Studio > Function Reverse Array.py > ...  
1  def reverse_array(arr):  
2      reversed_arr = []  
3      for i in range(len(arr) - 1, -1, -1):  
4          reversed_arr.append(arr[i])  
5      return reversed_arr  
6  
7  array = [1, 2, 3, 4, 5]  
8  print("Original array:", array)  
9  print("Reversed array:", reverse_array(array))
```

Output:

```
PS C:\Users\student> & "C:/Program F:  
Original array: [1, 2, 3, 4, 5]  
Reversed array: [5, 4, 3, 2, 1]  
PS C:\Users\student>
```

2. Use a stack to reverse a string

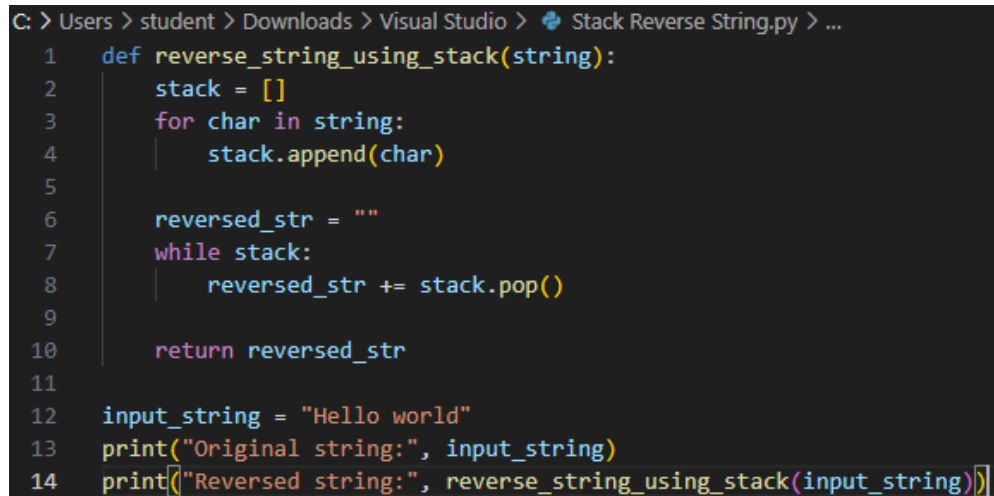
Code:

```
def reverse_string_using_stack(string):
    stack = []
    for char in string:
        stack.append(char)

    reversed_str = ""
    while stack:
        reversed_str += stack.pop()

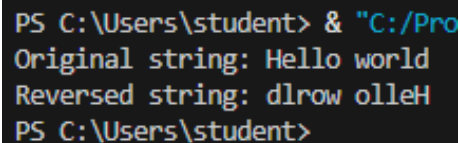
    return reversed_str

input_string = "Hello world"
print("Original string:", input_string)
print("Reversed string:", reverse_string_using_stack(input_string))
```



```
C: > Users > student > Downloads > Visual Studio > Stack Reverse String.py > ...
1  def reverse_string_using_stack(string):
2      stack = []
3      for char in string:
4          stack.append(char)
5
6      reversed_str = ""
7      while stack:
8          reversed_str += stack.pop()
9
10     return reversed_str
11
12     input_string = "Hello world"
13     print("Original string:", input_string)
14     print("Reversed string:", reverse_string_using_stack(input_string))
```

Output:



```
PS C:\Users\student> & "C:/Pro
Original string: Hello world
Reversed string: dlrow olleH
PS C:\Users\student>
```

3. Write a program that checks if a given expression has balanced parentheses using a stack.

Example:

a. Input: "(a + b) * (c - d)" → Output:

b. Balanced Input: "(a + b * (c - d)" → Output: Not Balanced

Code:

```
def is_balanced(expression):
```

```
    stack = []
```

```
    opening = "({["
```

```
    closing = ")}]"
```

```
    matches = {'(': ')', '[': ']', '{': '}': '[]'}
```

```
    for char in expression:
```

```
        if char in opening:
```

```
            stack.append(char)
```

```
        elif char in closing:
```

```
            if not stack or stack[-1] != matches[char]:
```

```
                return "Not Balanced"
```

```
            stack.pop()
```

```
    return "Balanced" if not stack else "Not Balanced"
```

```
expr1 = "(0 + 1) * (2 - 3)"
```

```
expr2 = "(0 + 1 * (2 - 3)"
```

```
print(expr1, "=", is_balanced(expr1))
```

```
print(expr2, "=", is_balanced(expr2))
```

```

1  def is_balanced(expression):
2      stack = []
3      opening = "({["
4      closing = ")}]"
5      matches = {'(': ')', '{': '}', '[': ']'}
6
7      for char in expression:
8          if char in opening:
9              stack.append(char)
10         elif char in closing:
11             if not stack or stack[-1] != matches[char]:
12                 return "Not Balanced"
13             stack.pop()
14         return "Balanced" if not stack else "Not Balanced"
15
16     expr1 = "(0 + 1) * (2 - 3)"
17     expr2 = "(0 + 1 * (2 - 3)"
18     print(expr1, "=", is_balanced(expr1))
19     print(expr2, "=", is_balanced(expr2))

```

Output:

```

PS C:\Users\student> & "C:/Program Files/Python37/python.exe" .\balanced.py
(0 + 1) * (2 - 3) = Balanced
(0 + 1 * (2 - 3) = Not Balanced
PS C:\Users\student>

```

4. Include all sources used as refence.

<https://www.geeksforgeeks.org/python/check-for-balanced-parentheses-in-python/>

<https://runestone.academy/ns/books/published/pythonds/BasicDS/SimpleBalancedParentheses.html>

<https://algo.monster/liteproblems/1190>

<https://www.topcoder.com/thrive/articles/generating-and-checking-balanced-parentheses>

<https://www.geeksforgeeks.org/dsa/check-for-balanced-parentheses-in-an-expression/>
<https://takeuforward.org/data-structure/check-for-balanced-parentheses/>